

Преобразование структурной схемы надёжности в граф состояний и марковскую модель

0) Цель

Разработать программу, в которой пользователь вручную создаёт структурную схему надёжности с поддержкой вложенных подсхем до 4 уровней. На основе схемы система автоматически строит:

1. граф состояний и переходов для текущего уровня;
2. матрицу интенсивностей переходов Q по заданным интенсивностям отказов λ
3. распределение вероятностей по состояниям на i -ом шаге/этапе по марковской модели с учётом режимов эксплуатации и длительностей этапов.

1) Редактор структурной схемы надёжности

1.1 Ручное создание схемы

Пользователь должен иметь возможность создавать схему вручную:

- добавлять блоки (элементы);
- соединять их связями;
- задавать тип структуры узла:
 - последовательная;
 - параллельная;
 - k -из- n ;
 - листовой элемент(на самом глубоком уровне).

1.2 Атрибуты элемента

Для каждого листового элемента задаются:

- уникальный идентификатор и имя;
- интенсивности отказов λ по режимам (см. раздел [\[sec:lamdas\]](#));
- принадлежность группе одинаковых элементов.

Для композитного элемента задаются:

- уникальный идентификатор и имя;
- ссылка на внутреннюю подсхему (подуровень).

2) Вложенность схемы (до 4 уровней)

2.1 Общая логика

Любой блок может быть композитным и содержать вложенную структурную схему следующего уровня. Максимальная глубина вложенности: 4 уровня.

2.2 Навигация

Должны поддерживаться:

- переход внутрь композитного элемента по клику;
- breadcrumb-навигация и кнопка “на уровень выше”;
- запрет перехода глубже 4 уровней.

2.3 Связь с графом состояний

При переходе на другой уровень:

- отображается схема выбранного уровня;
- граф состояний и переходов пересчитывается и отображается для текущего уровня.

3) Построение графа состояний и переходов

3.1 Определение состояния

Состояние характеризует, какие элементы (или подструктуры) на текущем уровне находятся в состоянии:

- **исправно;**
- **частичный отказ;**
- **отказ.**

3.2 Критерий работоспособности

Для каждого состояния система должна определять работоспособность верхней структуры текущего уровня:

- для последовательной структуры: исправно, если все элементы исправны;
- для параллельной структуры: исправно, если исправен хотя бы один элемент;
- для k -из- n : исправно, если число работоспособных элементов $\geq k$.

3.3 Переходы

Переход соответствует отказу одного элемента за малый интервал времени. В базовом варианте модель невосстанавливаемая (без ремонта): переходы идут только в сторону ухудшения. Самопереходы $S_i \rightarrow S_i$ допускается не отображать графически, но они учитываются при формировании матриц.

4) Интенсивности отказов по этапам (режимам)

4.1 Типы этапов

Поддерживаются четыре типа этапов:

1. транспортирование;

2. хранение;
3. функционирование;
4. выключенное состояние (пауза).

4.2 Задание λ по этапам

Для каждого листового элемента должны поддерживаться способы задания λ :

Для транспортирования, выключенного состояния и функционирования:

- либо можно задать самому значение λ для каждого элемента на этапе;
- либо выражение через λ другого этапа, например:

$$\lambda_{\text{тр}} = 0.5\lambda_{\text{эф}}, \quad \lambda_{\text{выкл}} = 0.2\lambda_{\text{эф}}.$$

Для хранения:

$$\lambda_{\text{хр}} = 0.1 \cdot \lambda_{\text{эф}},$$

4.3 Длительность этапов

Пользователь должен задавать длительность каждого этапа T_k (в согласованных единицах времени).

5) Формирование матрицы интенсивностей Q

5.1 Определение

Для каждого этапа k формируется матрица интенсивностей переходов (generator) $Q^{(k)}$, где:

- $Q_{ij}^{(k)}$ при $i \neq j$ — интенсивность перехода из состояния i в состояние j на этапе k ;
- $Q_{ii}^{(k)}$ — диагональный элемент, определяемый условием нулевой суммы по строке:

$$Q_{ii}^{(k)} = - \sum_{j \neq i} Q_{ij}^{(k)}.$$

5.2 Формирование внедиагональных элементов

Для $i \neq j$ величина $Q_{ij}^{(k)}$ равна сумме интенсивностей отказов тех элементов, отказ которых переводит систему из состояния i в состояние j на этапе k .

5.3 Разные этапы — разные матрицы

Поскольку λ зависит от режима, матрицы $Q^{(k)}$ для разных этапов могут различаться.

6) Расчёт вероятностей состояний на i -ом шаге

6.1 Входные данные

На вход подаются начальные вероятности состояний на первом этапе:

$$\mathbf{p}^{(0)}.$$

Обычно это $\mathbf{p}^{(0)} = (1, 0, \dots, 0)$, но должен поддерживаться произвольный ввод.

6.2 Требуемый результат

Требуется вычислить вероятность находиться в каждом состоянии после i -го шага (последовательности этапов циклограммы):

$$\mathbf{p}^{(1)}, \mathbf{p}^{(2)}, \dots, \mathbf{p}^{(i)}.$$

6.3 Расчёт по марковской модели

Сначала вычисляется:

$$P^{(k)} = e^{-Q^{(k)}T_k}$$

после чего:

$$\mathbf{p}^{(k+1)} = \mathbf{p}^{(k)}P^{(k)}.$$

7) Интерфейс и вывод результатов

На каждом уровне вложенности пользователь должен видеть:

- редактор текущей структурной схемы;
- граф состояний и переходов для текущего уровня;

И в каком нибудь выдвижном окошке

- матрицы $Q^{(k)}$ и/или $P^{(k)}$ по этапам;
- таблицу вероятностей $\mathbf{p}^{(i)}$ для выбранного i или по всем этапам.

Должны быть предусмотрены экспорт/импорт:

- схемы (например, в формате JSON);
- результатов расчёта (CSV/Excel/JSON).